Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO4 – Assessment Tool 1: Coding Challenge – Cloud Release Budget Planner
Course Outcome Assessed	Solve optimization problems using dynamic programming and greedy strategies.
Duration	In-class (one session)
Mode	Individual – conducted in class
Student Reg. No.	192511308
Student Name	Divya Dharshini V S

Instructions: This is an in-class coding challenge. Solve it individually in C, C++, Java or Python during the session. Your program must read from standard input and print exactly the output format shown. Before the session ends, push your code to your GitHub repository and share the link through Google Classroom. Hidden test cases, including edge cases and a large input, will be used for evaluation, and you may be asked to explain your code.

CODING CHALLENGE TASK

Problem – Cloud Release Budget Planner: A SaaS company is planning its next monthly release. The DevOps team has proposed n microservice features. Deploying feature i consumes cost[i] units of the monthly cloud budget (compute, storage and testing hours) and delivers business value value[i], estimated by the product team. A feature is either deployed completely or not at all; it cannot be partly deployed or deployed twice. The total cost of the deployed features must not exceed the budget W. Write a program that selects the features that maximise the total business value, and also reports the value a simple greedy rule would have achieved, so the team can see whether planning by "value per unit cost" is safe.

Input Format: The first line contains n and W. Each of the next n lines contains a feature name (a single word), its cost and its value.

Output Format: Exactly four lines: Greedy Value = G, Maximum Value = V, Selected = the chosen feature names in input order separated by single spaces (or NONE), and Budget Used = B.

Tie-Breaking Rule: If several selections give the maximum value, report the one with the smallest Budget Used; any selection with that value and budget is accepted.

Constraints: $1 \le n \le 200$; $0 \le W \le 10{,}000$; $1 \le cost[i] \le 1{,}000$; $0 \le value[i] \le 1{,}000$. Time limit: 2 seconds per test.

Sample 1:

```
Input:
3 50
Auth 10 60
Search 20 100
Payments 30 120

Output:
Greedy Value = 160
Maximum Value = 220
Selected = Search Payments
Budget Used = 50
```

Sample 2:

```
Input:
6 15
Cache 7 60
Search 4 36
Chat 8 33
Reports 5 34
Alerts 2 47
Backup 6 63

Output:
Greedy Value = 146
Maximum Value = 170
Selected = Cache Alerts Backup
Budget Used = 15
```

Tasks:

(a) Model the Problem and Test the Greedy Intuition: State the input, output, objective and constraint. Implement the greedy rule "take services in decreasing order of value per unit cost while they fit" and explain, using Sample 1, why it does not always give the optimum.

(b) Design the Dynamic Programming Solution: Define the DP state precisely, write the recurrence with its base cases, and implement the optimal solution so that it meets the constraints and time limit. Explain the order in which the table is filled.

(c) Reconstruct the Release Plan: Extend your solution to output the selected services in input order and the budget used, following the tie-breaking rule. Print NONE when no service is selected.

(d) Test and Analyse: Run both samples and your own edge cases (zero budget, no service fits, every service fits, tied optima, a single service) and a large test at the maximum constraints. State the time and space complexity and explain why the DP is called pseudo-polynomial.

Rubrics – Coding Challenge Task

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Mark Evaluation:

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20				
Algorithm	25				
Code Implementation	20				
Competitive Performance	20				
Test Case Handling	15				
Total Marks					

Code of Conduct

I Divya Dharshini V S (192511308) (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Date: _____

CO4 – ASSESSMENT TOOL 1

CODING CHALLENGE – CLOUD RELEASE BUDGET PLANNER

(a) Model the Problem and Test the Greedy Intuition

1. Problem Understanding

The Cloud Release Budget Planner is a 0/1 Knapsack optimization problem. A company has n microservice features, each having a deployment cost and business value. The objective is to select features that maximize the total business value without exceeding the available cloud budget.

2. Input, Output, Objective and Constraint

Input:

- Number of features (n).
- Available cloud budget (W).
- Feature name, cost and business value.

Output:

- Greedy Value (G).
- Maximum Value (V).
- Selected feature names.
- Total Budget Used (B).

Objective Function:

Maximize:

$V = \Sigma \text{ value}[i] \times x[i]$

Subject to:

$\Sigma \text{ cost}[i] \times x[i] \leq W$

Where:

x[i] = 1, if feature i is selected.

x[i] = 0, otherwise.

The selection is binary because each feature must be deployed completely or not deployed.

3. Greedy Algorithm

The greedy approach selects features based on decreasing value per unit cost.

Value-to-Cost Ratio:

Ratio[i] = value[i] / cost[i]

Algorithm:

1. START.
2. Read n, W and all feature details.
3. Calculate the value-to-cost ratio of each feature.
4. Sort features in decreasing order of their ratios.
5. Initialize used budget and total value to zero.
6. Traverse the sorted features.
7. If a feature fits within the remaining budget, select it.
8. Update the used budget and total value.
9. Print the greedy value.
10. STOP.

4. Greedy Analysis Using Sample 1

Input:

3 50

Auth 10 60

Search 20 100

Payments 30 120

Calculate the ratios:

Feature	Cost	Value	Ratio
Auth	10	60	6.00
Search	20	100	5.00
Payments	30	120	4.00

Greedy selection:

Step	Feature	Cost	Value	Budget Used
1	Auth	10	60	10
2	Search	20	100	30
3	Payments	30	Not selected	30

Payments cannot be selected because only 20 budget units remain.

Therefore,

Greedy Value = 60 + 100 = **160**

However, selecting Search and Payments gives:

Total Cost = 20 + 30 = 50

Total Value = 100 + 120 = **220**

Thus, the greedy method produces 160, whereas the optimal solution produces 220.

Conclusion:

Greedy selection does not always produce the optimal solution because choosing a locally best ratio may prevent a better overall combination.

(b) Design the Dynamic Programming Solution

1. Dynamic Programming Approach

Dynamic Programming solves the optimization problem by dividing it into smaller overlapping subproblems and storing their results in a table. Since every feature is either selected or rejected, the problem follows the 0/1 Knapsack principle.

2. DP State Definition

Let: DP[i][w] = Maximum business value obtained using the first i features with a budget capacity of w.

Where:

- i represents the number of considered features.
- w represents the available budget.
- DP[i][w] stores the maximum achievable value.

3. Mathematical Recurrence

For each feature, there are two possibilities:

Case 1: Feature is not selected

DP[i][w] = DP[i-1][w]

Case 2: Feature is selected

DP[i][w] = value[i] + DP[i-1][w-cost[i]]

This is possible only when:

cost[i] ≤ w

Therefore, the recurrence is:

DP[i][w] = max(DP[i-1][w], value[i] + DP[i-1][w-cost[i]])

If cost[i] > w:

DP[i][w] = DP[i-1][w]

4. Base Cases

DP[0][w] = 0

DP[i][0] = 0

Explanation:

- With zero features, the maximum value is zero.
- With zero budget, no feature can be selected because all costs are positive.

5. Dynamic Programming Algorithm

1. START.
2. Read the number of features and budget.
3. Initialize the DP table with zeros.
4. Traverse features from 1 to n.
5. Traverse budget capacities from 0 to W.
6. If the current feature exceeds the capacity, copy the previous result.
7. Otherwise, calculate both selection possibilities.
8. Store the maximum value in the DP table.
9. Continue until all features and capacities are processed.
10. Return DP[n][W].
11. STOP.

6. Short Pseudocode

```
BEGIN
  FOR i = 1 TO n
    FOR w = 0 TO W
      DP[i][w] = DP[i-1][w]
```

```
        IF cost[i] <= w
            DP[i][w] = MAX(DP[i][w],
                value[i] + DP[i-1][w-cost[i]])
    OUTPUT DP[n][W]
END
```

7. Order of Table Filling

The DP table is filled row by row.

- First, initialize the zero-feature row.
- Process the first feature for every budget.
- Process the second feature using the previous row.
- Continue until all n features are processed.
- The final optimal value is obtained from DP[n][W].

The previous row is used instead of the current row to ensure that each feature is selected at most once.

8. Worked Example

Feature	Cost	Value
Auth	10	60
Search	20	100
Payments	30	120

Available Budget = 50.

Important DP calculations:

For Auth:

DP[1][10] = max(0, 60) = 60

For Search:

DP[2][30] = max(60, 100 + 60)

DP[2][30] = 160

For Payments:

DP[3][50] = max(160, 120 + 100)

DP[3][50] = 220

Therefore: Maximum Business Value = 220

The optimal combination is Search and Payments.

(c) Reconstruct the Release Plan

1. Solution Reconstruction

Finding the maximum value alone is not sufficient. The company must also know which microservices should be deployed. Therefore, the DP table is used to reconstruct the selected features.

2. Reconstruction Procedure

1. Start from DP[n][W].
2. Find the smallest budget capacity that achieves the maximum value.
3. Traverse the DP table backwards from feature n.
4. If DP[i][w] differs from DP[i-1][w], select feature i.
5. Subtract its cost from the remaining budget.
6. Otherwise, skip that feature.
7. Continue until all features are checked.
8. Display the selected features in their original input order.

3. Tie-Breaking Rule

If multiple selections produce the same maximum value, the selection with the smallest actual budget usage must be reported.

To implement this:

Find the smallest w such that:

DP[n][w] = DP[n][W]

This capacity represents the minimum budget needed to achieve the maximum value.

During reconstruction, equal DP values are resolved by skipping the current feature. This produces a valid optimal selection without increasing the required budget.

4. Selected Feature Example

For Sample 1:

Feature	Cost	Value	Selection
Auth	10	60	No
Search	20	100	Yes
Payments	30	120	Yes

Total Budget Used:

20 + 30 = 50

Maximum Value:

100 + 120 = 220

Selected = Search Payments

Budget Used = 50

If no feature is selected, the program prints NONE.

(d) Test and Analyse

1. Sample Test Case 1

Input:

3 50

Auth 10 60

Search 20 100

Payments 30 120

Output:

Greedy Value = 160

Maximum Value = 220

Selected = Search Payments

Budget Used = 50

2. Sample Test Case 2

Input:

6 15

Cache 7 60

Search 4 36

Chat 8 33

Reports 5 34

Alerts 2 47

Backup 6 63

Greedy Calculation:

The decreasing ratio order is Alerts, Backup, Search, Cache, Reports and Chat.

Selected features:

Alerts + Backup + Search

Total Cost = 2 + 6 + 4 = 12

Total Value = 47 + 63 + 36 = 146

Optimal Selection:

Cache + Alerts + Backup

Total Cost = 7 + 2 + 6 = 15

Total Value = 60 + 47 + 63 = 170

Output:

Greedy Value = 146

Maximum Value = 170

Selected = Cache Alerts Backup

Budget Used = 15

3. Edge Case Testing

Case 1: Zero Budget

Input:

2 0

A 1 10

B 2 20

Output:

Greedy Value = 0

Maximum Value = 0

Selected = NONE

Budget Used = 0

Explanation: No feature can be selected because the available budget is zero.

Case 2: No Service Fits

Input:

2 3

A 4 10

B 5 20

Output:

Greedy Value = 0

Maximum Value = 0

Selected = NONE

Budget Used = 0

Explanation: Both features exceed the available budget.

Case 3: Every Service Fits

Input:

3 10

A 2 5

B 3 8

C 4 9

Output:

Greedy Value = 22

Maximum Value = 22

Selected = A B C

Budget Used = 9

Explanation: All three features can be deployed within the available budget.

Case 4: Tied Optimal Values

Input:

3 5

A 2 10

B 3 10

C 4 20

Output:

Greedy Value = 20

Maximum Value = 20

Selected = C

Budget Used = 4

Explanation: Selecting A and B gives value 20 using budget 5. Selecting C gives the same value using budget 4. Therefore, C is selected according to the tie-breaking rule.

Case 5: Single Service

Input:

1 5

Only 3 7

Output:

Greedy Value = 7

Maximum Value = 7

Selected = Only

Budget Used = 3

Explanation: The only available feature fits within the budget.

4. Maximum Constraint Testing

Consider:

n = 200

W = 10,000

Each feature has:

Cost = 1,000

Value = 1,000

The maximum number of features that can be selected is:

10,000 / 1,000 = 10

Maximum Value = 10 × 1,000 = 10,000

The DP algorithm processes:

200 × 10,001 = 2,000,200 states.

Expected output:

Greedy Value = 10000

Maximum Value = 10000

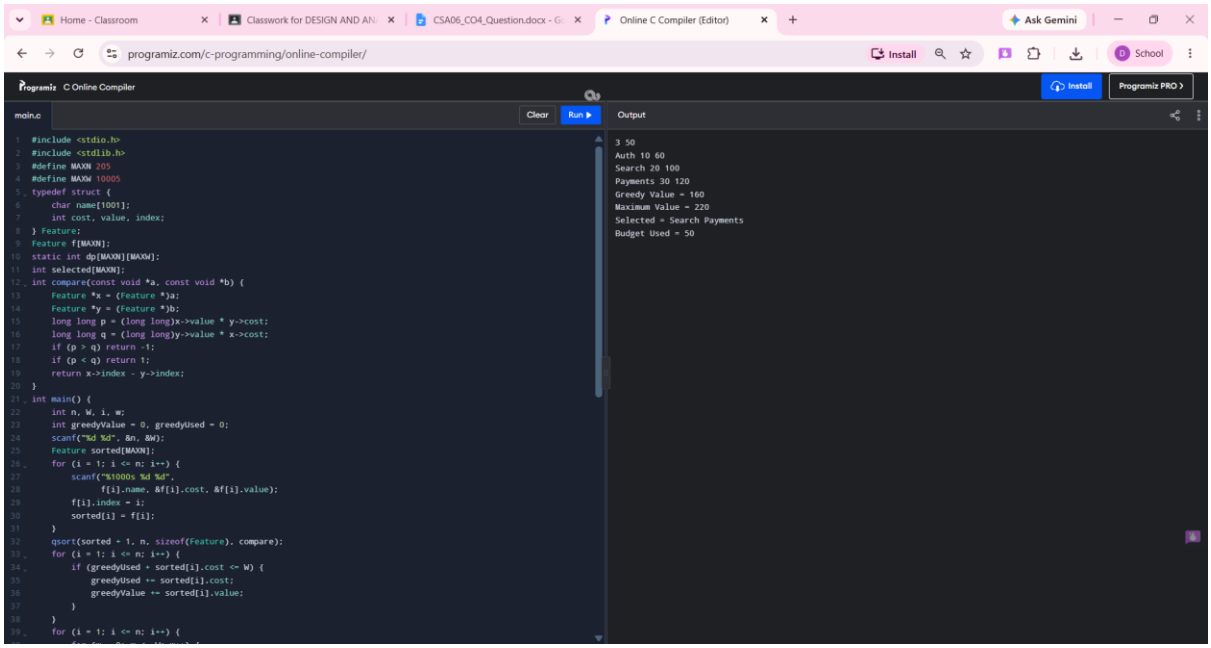
Selected = F1 F2 F3 F4 F5 F6 F7 F8 F9 F10

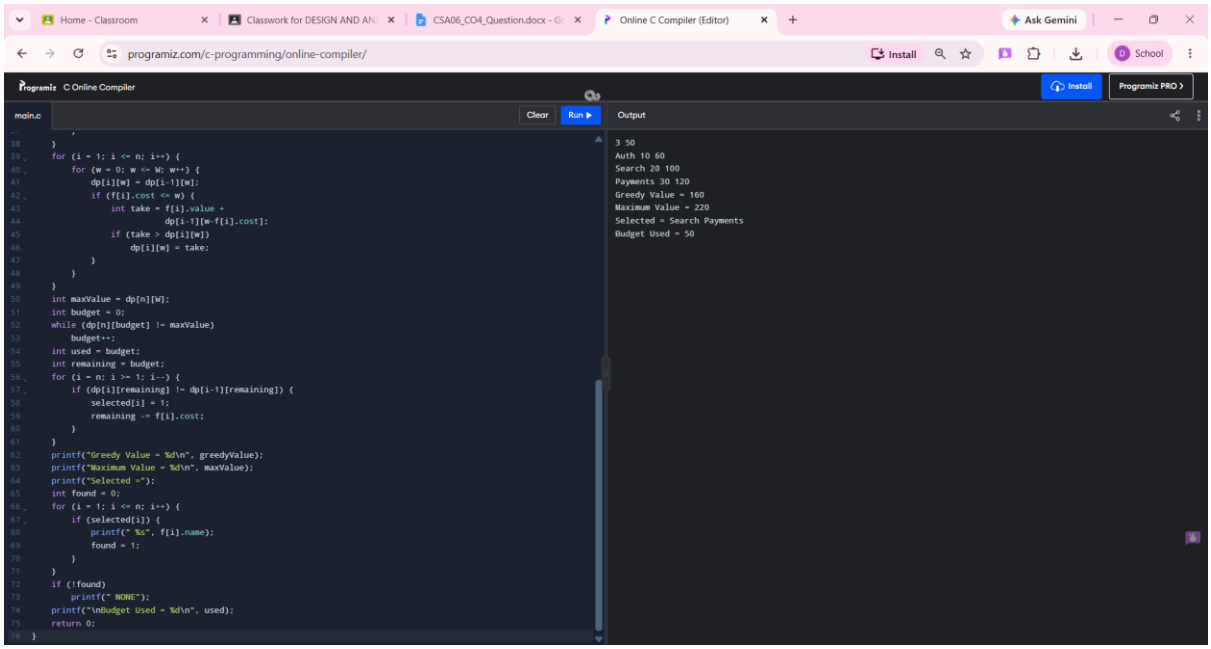
Budget Used = 10000

The implementation handles the maximum constraints using an iterative DP table without recursion.

5. Complete C Program

The following program implements greedy selection, dynamic programming, optimal feature reconstruction and minimum-budget tie-breaking.





6. Code Explanation

Greedy Sorting:

The compare() function sorts features in decreasing order of value-to-cost ratio. Cross multiplication avoids floating-point precision errors.

Greedy Selection:

The program selects each feature only when its cost fits within the remaining budget. It updates the greedy value and used budget.

DP Table Construction:

The nested loops calculate the maximum achievable value for every feature and budget combination.

Reconstruction:

The program identifies the smallest budget that achieves the maximum value and traces the DP table backwards to identify selected features.

Input Order:

The original feature array is maintained separately from the sorted greedy array. Therefore, selected features are printed in their original input order.

Output Handling:

The program prints exactly four lines. If no feature is selected, it prints NONE.

7. Time Complexity Analysis

Let:

n = Number of features.

W = Available budget.

A. Greedy Sorting

The features are sorted using qsort().

$T(n) = O(n \log n)$

Traversing the sorted features requires:

$T(n) = O(n)$

Therefore, greedy complexity:

$O(n \log n)$

B. Dynamic Programming

The DP table contains $(n + 1) \times (W + 1)$ states.

Each state requires constant-time operations.

Number of states:

$T(n, W) = n \times W$

Therefore:

$T(n, W) = O(nW)$

C. Reconstruction

Finding the minimum budget requires at most $W + 1$ comparisons.

Backtracking examines n features.

$T(n, W) = O(W + n)$

Total Time Complexity:

$T(n, W) = O(n \log n) + O(nW) + O(W + n)$

Therefore:

$T(n, W) = O(nW)$

The DP computation dominates the overall running time.

8. Space Complexity Analysis

The DP table requires:

$S(n, W) = (n + 1) \times (W + 1)$

Therefore:

$S(n, W) = O(nW)$

Additional storage includes:

- Feature arrays: O(n)
- Greedy sorted array: O(n)
- Selected array: O(n)
- Loop variables: O(1)

Therefore:

Total Space Complexity = O(nW)

The implementation uses iterative DP and does not require a recursion stack.

9. Why is Dynamic Programming Pseudo-Polynomial?

Dynamic Programming is called pseudo-polynomial because its running time depends on the numerical value of the budget W rather than the number of bits required to represent W.

The algorithm takes:

O(nW)

However, representing W in binary requires only:

O(log W) bits.

Therefore, the running time is polynomial in the numerical budget but not necessarily polynomial in the input size.

For example, if W becomes very large, the DP table also becomes large even though the binary representation of W requires relatively few bits.

Hence, 0/1 Knapsack using this DP approach is pseudo-polynomial.

10. Applications

1. Cloud resource allocation.
2. Software feature deployment planning.
3. Project budget optimization.

4. Server capacity management.
5. Storage and resource allocation.

11. Advantages of the Proposed Solution

The proposed system provides a reliable way to make cloud deployment decisions within a limited budget. By comparing Greedy and Dynamic Programming results, the DevOps team can understand the difference between quick selection and optimal planning. The reconstruction method identifies the exact features to deploy while minimizing budget usage when multiple optimal solutions exist. This helps organizations improve resource utilization, maximize business value and make better release management decisions.

12. Practical Significance

This approach can be extended to real-world cloud environments where multiple services compete for limited computing resources. It helps the DevOps team prioritize important features, control deployment expenses and avoid unnecessary resource usage. The optimal selection also improves planning accuracy by considering all possible feature combinations within the given budget. Thus, the system supports cost-effective, value-driven and efficient software release management.

13. Conclusion

The Cloud Release Budget Planner uses Greedy and Dynamic Programming to compare approximate and optimal feature selection. Greedy provides a quick value-based selection, while Dynamic Programming guarantees the maximum business value within the available budget. The reconstruction technique also ensures minimum budget usage when optimal values are tied. Thus, the proposed solution provides an efficient and reliable approach to cloud release planning.